```
`timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"

module alu (
    input wire [31:0] i_op1,       // Operand 1
    input wire [31:0] i_op2,       // Operand 2
    input wire [3:0] i_AluCtrl,    // ALU control signal
    output reg [31:0] o_Result     // ALU result
);

always @(*) begin
    case (i_AluCtrl)
        `ADD:   o_Result = $signed(i_op1) + $signed(i_op2);
        `SUB:   o_Result = $signed(i_op1) - $signed(i_op2);
        `AND:   o_Result = i_op1 & i_op2;
        `OR:    o_Result = i_op1 | i_op2;
        `XOR:   o_Result = i_op1 ^ i_op2;
        `SRL:   o_Result = i_op1 >> i_op2[4:0]; // Use only the lower 5 bits for
shifting
        `SLL:   o_Result = i_op1 << i_op2[4:0];
        `SRA:   o_Result = $signed(i_op1) >>> i_op2[4:0];
        `BUF:   o_Result = $signed(i_op2);
        `SLT:   o_Result = (i_op1[31] ^ i_op2[31]) ? {31'd0, i_op1[31]} : {31'd0,
$signed(i_op1) < $signed(i_op2)};
        `SLTU:  o_Result = {31'd0, i_op1 < i_op2};
        `EQ:    o_Result = {31'd0, (i_op1 == i_op2)};
        `GE:    o_Result = (i_op1[31] ^ i_op2[31]) ? {31'd0, i_op2[31]} : {31'd0,
($signed(i_op1) >= $signed(i_op2))};
        `GEU:   o_Result = {31'd0, (i_op1 >= i_op2)};

        // Atomic Operations
        `AMO_ADD:   o_Result = i_op1 + i_op2;
        `AMO_AND:   o_Result = i_op1 & i_op2;
        `AMO_OR:    o_Result = i_op1 | i_op2;
        `AMO_XOR:   o_Result = i_op1 ^ i_op2;
        `AMO_MIN:   o_Result = ($signed(i_op1) < $signed(i_op2)) ? i_op1 : i_op2;
        `AMO_MAX:   o_Result = ($signed(i_op1) > $signed(i_op2)) ? i_op1 : i_op2;
        `AMO_MINU:  o_Result = (i_op1 < i_op2) ? i_op1 : i_op2;
        `AMO_MAXU:  o_Result = (i_op1 > i_op2) ? i_op1 : i_op2;
        `AMO_SWAP:  o_Result = i_op2; // Swap operation just returns operand 2
        `AMO_LR:    o_Result = i_op1; // Load-Reserved returns the original value
        `AMO_SC:    o_Result = (i_op1 == i_op2) ? 32'b0 : 32'b1; //
Store-Conditional success (0) or failure (1)

        default: o_Result = 32'd0;
```

```
    endcase
end

endmodule
```